

AnchorOverlap: 基于消费者依赖的高效集合通信-GEMM 优化 (中文审阅版)

Anonymous Author(s)

Abstract

大模型张量并行执行中, 集合通信与矩阵乘法 (GEMM) 在关键路径上反复交替。为了重叠二者的执行, 已有方法对中间数据做划分, 并通过依赖调度和信号实现细粒度数据交接, 使下游任务在部分输入可用后即可开始。然而, 这些方法并不保证消费者所需的数据被优先生成。同时, 若把每个尚未消费的分块都保留在暂存中, 暂存占用会随工作量增长。我们提出 *AnchorOverlap*, 一套面向集合通信-GEMM 链路的运行时执行协议。该协议依据消费者的读取依赖确定交接粒度与数据生产顺序。在 AllGather-GEMM 中, 本地分片上的计算立即开始, 依赖远端行分块的任务随行分块到达而执行。在 GEMM-ReduceScatter 与 GEMM-AllReduce 中, 执行围绕使归约尽早启动的输出前沿组织, 前者优先生成构成该前沿的 GEMM 输出分块, 后者按逻辑顺序生成输出分块并在每个分块计算完成后立即启动递归倍增归约。在两条归约链路中, 这些输出分块被循环映射到活动窗口内有界的对称暂存槽位集合。到达与释放状态协调数据交接, 每个暂存槽位在消费者读完其中数据后即可复用。在 NVIDIA A100 与海光 K100_AI 平台上, *AnchorOverlap* 取得最高 1.40 $\times$ 的算子级加速, 并使 Llama3-70B 与 Qwen2.5-72B 的端到端预填充获得 1.02–1.22 $\times$ 加速。在三个 GEMM-ReduceScatter 窗口实验中, 相对全部分块驻留所用的窗口大小, 将活动窗口减半可使对称暂存占用降低 50%, 中位延迟仅增加 0.4%–0.6%。

1 引言

随着模型规模和上下文长度持续增长, DeepSeek-V4 [8]、Kimi K3 [16] 和 Qwen3.8 [32] 已报告万亿参数或百万 token 级的配置, 在单个 GPU 上部署这些生成式模型已不可行。张量并行 [35]、流水线并行 [23] 和数据并行划分参数, 支撑大规模训练 [14] 与推理服务 [18, 27, 46]。由此产生的 AllGather (AG)、ReduceScatter (RS) 和 AllReduce (AR) 等集合通信原语, 会与矩阵乘法 (GEMM) 在关键路径上反复交替。计算与通信重叠是缓解这一开销的重要手段。如图 1 所示, 每个 Transformer 层都重复经历 GEMM 产出部分结果、集合通信交接数据、下游消费的过程。在行切分激活下, Attention 和 MLP 的输入投影通过 AG-GEMM 获取并消费输入, 输出投影通过 GEMM-RS 归约部分结果并恢复行切分布局。在复制激活下, 输出投影改由 GEMM-AR 合并部分结果。两条执行路径中, 通信时间都在关键路径上反复累积。更重要的是, 每条链路的两端构成一组闭合的局部生产者-消费者依赖, 上游输出就是下游输入, 消费者只要取得部分输入即可开始, 而不必等待整体完成。这一结构既是重叠的机会, 也限制了可行的控制范围。

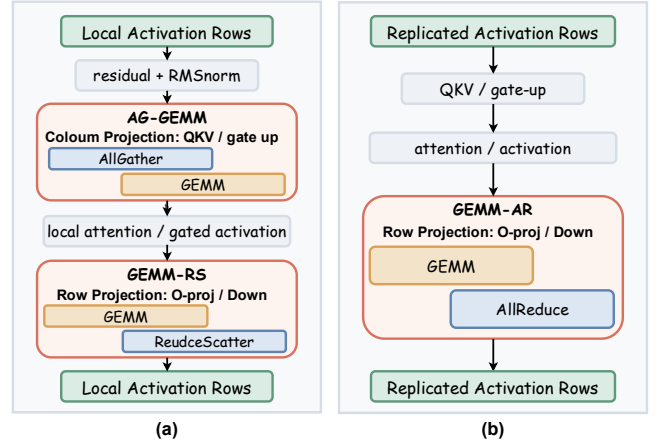


Figure 1. (a) 行切分激活。 (b) 复制激活。

已有工作通过任务分解 [13, 38]、内核融合或细粒度依赖 [4, 45]、信号或依赖感知调度 [12, 31] 来重叠相互依赖的计算与通信。重叠因此取决于交接边界和任务次序。在连续处理分块时, 消费者的提前启动还需满足两个条件, 消费者所需的跨 rank 数据要及时到达, 且接收位置上的旧数据已经消费完毕。我们把协调依赖驱动的数据供给与消费后的空间复用, 作为进一步改进重叠的切入点。

让消费者尽早启动需要回答两个相互关联的问题, 一次交接应提供哪些数据, 以及这些数据应在何时生成。对于一个选定的消费者任务, 交接边界应覆盖它实际读取的数据。边界过粗会使任务等待无关数据, 边界过细则会增加通知与调度开销。边界确定之后, 生产顺序决定这些读取依赖何时得到满足, 可能出现部分结果已发布就绪而消费者所需数据尚未生成的情况。以 GEMM-RS 为例, 归约一个输出分块需要来自各源 rank 的对应贡献。在图 2(a) 中, 一个源 rank 较晚完成其贡献, 其他输出的就绪通知无法使被选定的归约启动。图 2(b) 保持交接边界不变, 但优先生成构成当前归约前沿的贡献, 使目标归约更早与其余 GEMM 计算重叠。因此, 交接粒度和生产顺序需要围绕同一组消费者依赖组织, 才能把局部数据就绪转化为消费者的提前执行。

持续重叠还要求后续分块有可交接的空间。在单侧通信路径上, 写入对端可见暂存的数据必须保留到相关读取结束。为每个逻辑分块预留独立位置会使对称暂存占用随工作量增长, 而等整批任务完成后才统一回收空间又会引入妨碍持续重叠的等待。因此我们提出 *AnchorOverlap*, 一套面向集合通信-GEMM 链路的运行时执行协议, 它以消费者可独立处理的数据分块为共同边界, 把执行依赖与暂存生命周期衔接起来。在循环复

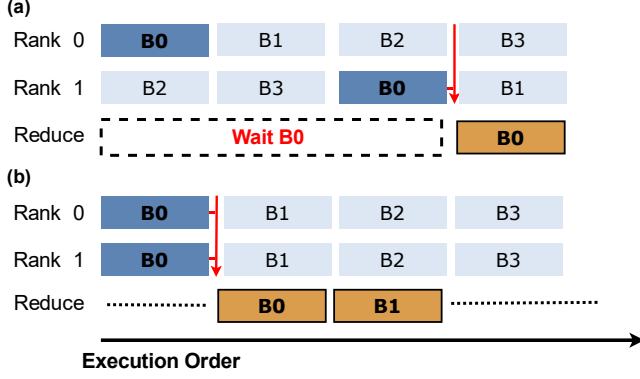


Figure 2. 生产顺序对归约启动的影响。(a) B0 较晚就绪。(b) 优先生成 B0 使其归约更早与其余 GEMM 重叠，随后归约 B1。

用暂存的归约路径上，消费完成后即允许向同一位置写入新数据，使执行在有限个物理槽位上持续推进。

AnchorOverlap 依据消费者依赖，对图 1 中的三条链路采用不同的执行策略。在 AG-GEMM 中，本地分片上的计算立即开始，依赖远端行分块的 GEMM 任务随行分块到达而启动。在 GEMM-RS 和 GEMM-AR 中，执行围绕使归约尽早启动的输出前沿组织。RS 优先生成构成该前沿的 GEMM 输出分块，使归约与其余输出的计算重叠。AR 逐块推进归约前沿，每个输出分块计算完成后立即启动递归倍增归约，使归约与后续分块的 GEMM 计算重叠。在 RS/AR 路径上，连续的输出分块复用活动窗口内的对称暂存槽位。当窗口深度、分块形状和参与设备数固定时，对称数据暂存容量不必随逻辑分块总数增长。

本文的贡献包括：

1. 一套面向集合通信-GEMM 链路的执行协议，依据消费者需要读取的数据确定交接粒度和数据生产顺序。它实例化为 AG 的本地与远端依赖驱动执行、RS 的前沿优先生产，以及 AR 的逐分块递归倍增归约。
2. 一套与数据交接相衔接的对称暂存复用机制。到达与释放状态约束数据的写入、读取和回收，使 RS/AR 能在有限个物理槽位上连续执行。
3. 我们在两个 GPU 平台上实现全部三条链路，并用配置搜索使执行适配不同负载。算子级和模型级实验评估重叠收益，消融实验考察就绪粒度、前沿优先生产，以及暂存容量与执行时延之间的权衡。

2 相关工作

已有研究通过分解与调度、融合内核、依赖信号和生产顺序等环节组织计算与通信重叠。我们按各类方法所控制的链路环节，把它们与 AnchorOverlap 对应起来。

2.1 分解与依赖感知调度

分解方法创建可调度的计算或张量单元，并把通信安排在其间。OCDC [39]、Domino [38] 和 CoCoNet [13] 切分相互依赖的计算，或在编译器与通信抽象之间协同优

化。Centauri [5]、CO2 [36]、Oases [20] 和 Async-TP [30] 调整通信划分或训练调度。ISO [41] 和 TokenWeave [10] 把这一思路扩展到序列与 token 维度，SARATHI 则采用分块预填充 [2]。

Syncopate [31] 构造生产者与消费者的依赖图，插入满足正确性所需的最少同步，使消费者可以在分块到达后立即处理，并通过调整 wave 顺序改善 AG-GEMM、GEMM-RS 和 GEMM-AR 的重叠。CommFuse [15] 把依赖通信的计算排在不依赖通信的工作之前，以缩短暴露的尾部。ConCCL [1] 利用 GPU 的 DMA 引擎并发执行通信与计算，Lee 等人 [19] 刻画了这种并发在性能与功耗上的影响。

这些结果表明，重叠粒度和并发度需要按负载调优，不能假定粒度越细性能就一定越好。若通信分组或内核划分与消费者的实际读取集合不匹配，过粗的交接会使本可执行的任务等待无关数据，过细的交接则增加通知与调度开销，却不一定更早满足消费者的全部依赖。

2.2 融合与细粒度内核依赖

这一族方法把通信与计算编入同一个内核，或通过细粒度依赖把它们绑定，使重叠发生在内核内部或紧邻的边界上。FLUX [4] 将通信融合进 GEMM 内核，cuBLASmp [24] 提供分布式 GEMM 库接口。T3 [28] 跟踪计算事件并透明地推动集合通信前进。TileLink [45] 提供 tile 为单位的生产者通知与消费者等待，由依赖决定消费者 tile 何时可执行。Comet [43] 沿消费者可直接使用的维度分解共享张量，并优先生成下游 MoE 可立即使用的部分。Iris [3] 在 Triton 中提供基于 tile 的对称内存抽象和工作组特化重叠。

这些方法已经把消费者依赖纳入粒度选择和执行调度。在集合通信-GEMM 链路中，消费者所需的数据还取决于张量在 rank 间的划分方式，一个 tile 的完成不一定满足全部跨 rank 读取依赖。采用单侧写入且接收端存储有限时，新数据还必须等待同一位置上的旧数据被读取之后才能写入。

2.3 生产者信号与重排

信号式方法利用内核内部的局部完成事件触发通信，使交接紧跟数据的生产。FlashOverlap [12] 以组内 tile 完成计数触发通信，把 tile 或 wave 分组，按执行顺序将每组输出重排为 NCCL 可连续处理的区域，通信后再恢复逻辑顺序。其分组搜索在较早的通信启动与较大消息的带宽收益之间权衡，使通信粒度和数据布局适应 GEMM 输出的完成进度。

针对 MoE 返回路径，Cui 等人 [7] 把 tile 级就绪信号与 segment 粒度通信结合起来。持久化 GEMM 内核优先生成发往远端的 tile，独立的通信内核在一个 segment 所需 tile 全部就绪后发出单侧传输。该方法预先准备目标 rank 和远端偏移等路由元数据来衔接生产与传输，并调节 segment 大小以在启动时间与传输带宽之间取得平衡。

FlashOverlap 按输出完成次序组织通信分组和布局，Cui 等人则优先生成待传输数据。二者共同说明，局部完成信号的收益取决于交接粒度和数据供给次序。

2.4 集合通信与分布式内核运行时

这一族工作提供集合通信算法、单侧通信原语与内核级的信号等待调度。TACCL [33]、MSCCLang [6]、Blink [37] 和 AutoCCL [42] 用于合成、描述或调优集合通信算法及其拓扑映射。NCCL [25]、NVSHMEM [26] 和 Triton-distributed [44] 提供通信、同步与设备端任务推进所需的执行接口。AnchorOverlap 保持 AG、RS 和 AR 的集合通信语义，控制这些操作产生的增量数据何时生成、消费和释放。

在数据暂存之外，GICC [34] 用单调递增的 epoch 复用有限的 NIC 状态和工作队列状态，X-Stage [40] 对任务发出后的有限容量和发送端反压建模。这些机制管理的是控制状态或尚未完成的操作。

流水并行 [22, 23] 与序列并行 [9, 17, 21] 重新安排训练的计算阶段，改变链路出现的位置与频率。Dist-Serve [46] 与 Splitwise [27] 将预填充和解码拆分到不同设备。vLLM [18] 与 vAttention [29] 改进服务端内存管理。FlashDecoding++ [11] 等工作优化解码注意力的执行。并行布局与部署选择可以改变通信发生的位置和数据规模，但这些外层决策本身并不规定一次调用内部分数据何时足以启动消费者，也不规定后续数据何时可以覆盖接收槽位。链路内部的依赖感知交接与这些方向互为补充。

3 系统设计

AG-GEMM、GEMM-RS 和 GEMM-AR 的通信语义并不相同，但共享同一组生产者-消费者关系，生产者逐步产生数据，消费者在所依赖的数据可用后即可执行下一个可执行任务。AnchorOverlap 把这种关系实现为一套运行时执行协议。本节详细说明 AnchorOverlap 的优化机制。

3.1 设计概览

在图 3(a) 的行切分布局中，本地输入立即进入本地快速 GEMM 路径，远端输入随 AllGather 到达后触发其依赖的 GEMM 任务。两条路径写入输出张量各自的行区间。门控激活随后送入输出投影，它在每个 rank 上生成本地对归约的贡献。RS 优先产出同一归约任务所需的输出分块，目的端可以汇集这些贡献并在生产者计算后续分块的同时开始归约。每个目的端保留归约后的行，得到行切分的输出。

在图 3(b) 的复制布局中，每个 rank 已持有完整激活，输入投影无需 AllGather。门控激活之后，输出投影逐块推进归约前沿，当前输出分块计算完成后立即启动递归倍增归约。Reduce1 和 reduce2 箭头展示四卡示例中的两次交换。当前分块被归约的同时 GEMM 计算后续分块，每个 rank 最终得到相同的完整输出。两种布局中，ready 和 reuse 箭头标记两类反复出现的转换，ready 使已完成的分块交接给其消费者，reuse 把已消费的暂存空间归还给后续分块。

对于包含 p 个 rank 的通信组， A_s 、 W_s 和 D_s 分别表示 rank s 的激活、权重和输出张量，GEMM 形状为 $M \times N \times K$ 。行切分的 AG 输入和 RS 输出各含 $m = M/p$ 个本地行。

Table 1. 设计中使用的符号。

符号	含义
p	通信组中的 rank 数。
s, d, u	源 rank、目的 rank 与远端 AG rank。
A_s, W_s, D_s	rank 本地的激活、权重和输出张量。
C_c, B_b	行分块与列带。
c, b	分块与列带索引。
$h_c, w_b, w_b^{\max}$	分块高度、带宽与最大带宽。
B	列带数量。
F	每个 RS 目的端的优先行分块数。
L	RS 活动窗口深度。
R_{RS}, R_{AR}	RS 归约流水线数与 AR 输出分块槽数。
q, r	RS 窗口槽位与 AR 输出分块槽位。
ω, x	逻辑数据块及其映射的物理槽位。
$\gamma, \tau_\omega, \pi_\omega$	RS 就绪轮次，当前与前驱物理槽位序号。

AR 中每个 rank 的 GEMM 产出 M 行局部结果，归约后每个 rank 保留全部 M 行输出。 C_c 和 B_b 分别表示一段连续的行分块和一个列带，其交叠区域以 (c, b) 索引，称为面板。图 3 右侧的 C_0/B_0 等标签标识这些输出分块。表 1 汇总了符号。

Ω 表示一次调用中的全部逻辑任务， $\mathcal{A}_t \subseteq \Omega$ 表示时刻 t 仍占用物理槽位的任务。AG 交接输入行分块。RS 和 AR 通过可复用的对称暂存槽位交接输出面板，并在消费者读取结束后释放对应槽位。

3.2 消费者启动条件与数据生产顺序

在 AG-GEMM 中，消费者能否启动取决于它读取的输入行。只依赖本地分片的任务可以立即进入本地 GEMM 路径。远端任务等待它实际读取的行分块 (u, c) 。跨越多个行分块的 GEMM tile 需等齐这些分块，但不会等待无关的远端输入。因此本地计算可以与 AllGather 重叠，已到达远端行分块上的计算可以与其余传输重叠。

RS 的消费者需要每个 rank 对同一输出区域的贡献。例如图 3(a) 中的 0/0 标识目的 rank 的第一个行分块和第一个列带。只要其中一份贡献尚未产生，完成其他区域并不能使该面板可归约。AnchorOverlap 把补齐这些依赖的生产者任务称为唤醒任务，把消费者启动后继续进行的计算称为剩余生产任务。为了尽早补齐这些依赖，多带前沿路径中的每个生产者先为所有目的端计算前 F 个行分块，再计算其余分块，使这些归约所需的贡献进入生产早期阶段。前沿内的 tile 保持独立可调度，归约按实际就绪情况启动，与其余 GEMM 工作重叠。

首次输出提交时间用于比较两种实现的启动行为。图 4 将该过程划分为生产者启动至就绪、就绪至消费者启动、消费者启动至首次提交三个区间。在三个记录形状上，前沿优先实现均早于均匀顺序实现提交首个输出。

AR 在每个 rank 上保留完整的归约输出。在图 3(b) 中，各 rank 共同推进当前等待归约的前沿分块。AnchorOverlap 在其生产任务结束后立即提交其归

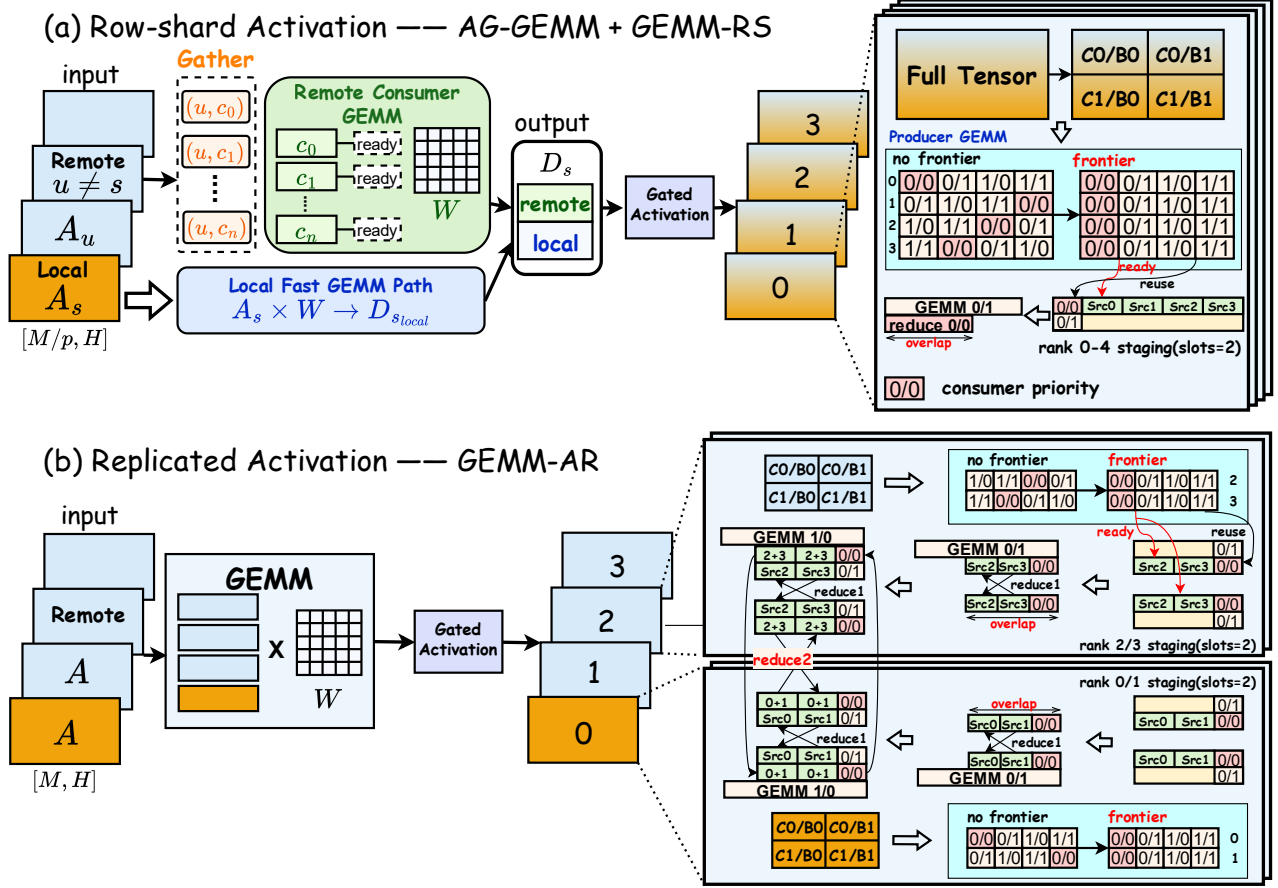


Figure 3. 两种张量并行布局下的 AnchorOverlap。(a) 行切分激活下的 AG-GEMM 与 GEMM-RS。(b) 复制激活下的 GEMM-AR。Ready 连接生产与消费，reuse 将已消费的槽位归还给后续分块。Reduce1 与 reduce2 是四卡 AR 的两次交换。

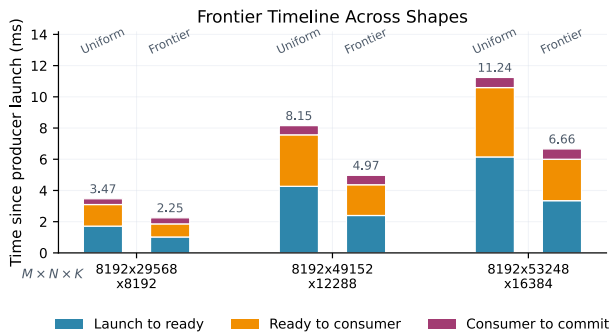


Figure 4. 均匀顺序与前沿优先实现的首次输出时间线。

约，使归约可以在结果就绪时启动并与后续分块的计算重叠。前沿在每个 rank 上按相同的行分块优先、列带次之的顺序推进。对于分块 0/0，reduce1 形成两卡之和，reduce2 将其合并为四卡结果，并与 0/1、1/0 等后续分块的 GEMM 重叠。RS 通过优先生产补齐归约依赖，AR

通过逐分块立即提交避免就绪结果被后续 GEMM 阻塞。二者都把启动与持续生产衔接起来。

3.3 基于活动窗口的对称暂存复用

RS 和 AR 都使用对称暂存接收来自其他 rank 的数据，RS 中是发往同一输出分块的本地贡献，AR 中是每轮递归倍增交换产生的累积结果。当前分块的全部读取和输出更新完成后，这块空间即可服务后续分块。AnchorOverlap 限制占用暂存槽位的任务数量，使逻辑分块在一个被复用的物理分配上持续推进。

图 5 展开说明这一复用机制。左侧为每个逻辑行分块预留独立区域，暂存随分块数量增长。右侧维护深度为 $L = 2$ 的窗口，容纳 C_k 和 C_{k+1} 。每个行分块划分为列带 B_0 和 B_1 ，每个面板的 SRC 区域接收来自相应源 rank 的贡献。 C_k 的一个面板被消费后， C_{k+2} 的对应面板即可使用同一物理位置， C_{k+1} 与 C_{k+3} 以相同方式复用位置。列带决定一个归约任务的覆盖范围，窗口深度决定同时拥有暂存空间的行分块数量。

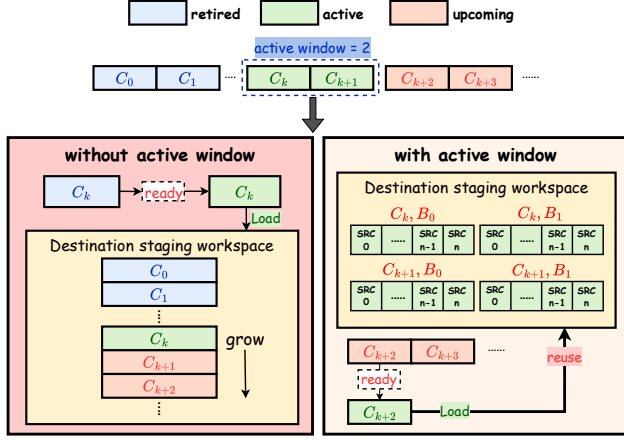


Figure 5. 活动窗口下的暂存布局与复用，以 RS 为例。两个行分块位置各含两个列带面板，每个面板位置在消费后可复用。

RS 将行分块 c 映射到窗口位置 $q = c \bmod L$ 并保留列带索引 b ，共得到 LB 个面板位置。AR 将逻辑分块 $k = cB + b$ 映射到 $r = k \bmod R_{AR}$ ，预留 R_{AR} 个输出分块槽位。在图 3(b) 中，一个分块在两次交换中持有同一槽位，在槽内用不同的源 rank 区域接收各轮数据。第二轮归约结束后该槽位可复用。 $\mathcal{A}_t$ 随分块完成消费和新分块进入而变化，并受这些槽位数量约束。

设分块高度为 h_c ，最大带宽为 $w_b^{\max}$ ，每个元素占 e 字节。为此配置分配时，RS 每个 rank 所需的数据暂存容量约为

$$M_{RS}^{\text{data}} \approx L \cdot B \cdot p \cdot h_c \cdot w_b^{\max} \cdot e. \quad (1)$$

每个面板位置为 p 个源 rank 各预留一个接收区域。本地贡献直接作为归约初值，节省一次暂存写入，但分配仍保留其源区域。AR 同样按源 rank 划分每个槽位，所需容量为

$$M_{AR}^{\text{data}} \approx R_{AR} \cdot p \cdot h_c \cdot w_b^{\max} \cdot e. \quad (2)$$

AR 每一轮读取属于其交换对端的区域。以上两式描述接收数据的对称暂存，本地 GEMM 结果与最终输出驻留在普通显存中。

分块尺寸、列带数、通信组规模和窗口深度固定后，增加逻辑行分块数量不需要额外的数据槽位。图 6 以 $L = 4$ 为例，比较按分布局计算得到的容量，活动窗口分配保持固定，而逐逻辑分块单独分配则随分块数量增长。图中的暂存包络表示为该配置预留的对称暂存容量。窗口过小会使后续传输等待空闲位置，窗口过大则增加内存占用，因此窗口深度还需结合执行时延选择。

有限槽位也要求一条何时可以安全复用的规则。在图 5 中，ready 表示当前源面板已生成，而复用要求目的端已读完同一位置上的旧面板。传输流只在两个条件同时满足后写入新数据，并在写入完成后发布到达状态。消费者等待当前任务所需的全部数据，完成读取和输出更新后发布释放状态。这一顺序保护未消费的数据不被后续面板写入覆盖。

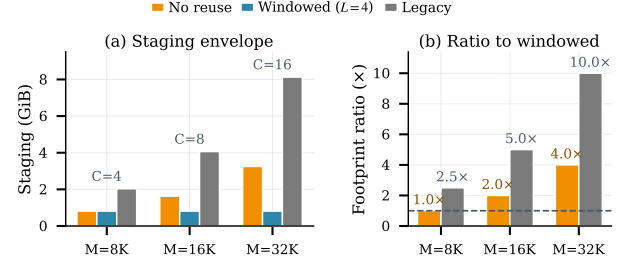


Figure 6. 固定 $N = 53248$ 、 $K = 16384$ 和 $L = 4$ 时对称暂存容量随逻辑行分块数量的变化。

由于一个槽位先后接收多个分块，到达与释放状态为每次使用携带序号。消费者等待当前分块的到达序号，传输流等待该槽位上一分块的释放序号。旧分块遗留的状态因此不会成为新分块的完成通知。RS 在面板归约后释放面板位置，AR 在全部递归倍增阶段结束后释放分块槽位。这一释放回授实现了图 3 中的复用路径，使生产与传输在有限暂存空间内持续进行。

4 算子实例与运行时调优

本节描述 AnchorOverlap 的实现。我们组合 Triton 消费者内核、单侧通信与对称内存原语，为每条链路提交计算、传输数据和执行归约。配置搜索为给定问题形状选择交接粒度和执行资源，以降低算子时延。

4.1 AG-GEMM 实例

rank s 持有激活分片 A_s 和权重分片 W_s 。AllGather 提供其他 rank 的激活分片，输出按输入来源划分：

$$D_{s,u} = A_u W_s, \quad u \in [0, p). \quad (3)$$

本地 GEMM 直接读取 A_s 并写入 $D_{s,s}$ 。与此同时，AllGather 将其他 rank 的输入复制进工作区，远端 GEMM 从工作区消费陆续到达的行分块并写入对应的 $D_{s,u}$ 。两条计算路径使用独立的数据流并写入互不重叠的输出行，与图 3(a) 中的本地快速 GEMM 路径和远端消费者 GEMM 一致。

算法 1 给出行分块交接路径。AnchorOverlap 用 InitReady 在调用者流上将就绪数组清零，并让通信流等待该初始化完成。远端消费者在调用者流上运行，本地计算流同样等待输入准备完成。一次传输完成后，仅当复制的数据可读时才发布 $\text{ready}[u, c] = 1$ 。每个远端 GEMM tile 依据它读取的输入行推导自身依赖，并在计算前等齐这些行分块。就绪的 tile 可以在 AllGather 其余部分仍在进行时继续执行。当通信路径或消费者不支持行分块信号时，运行时改用 rank 粒度的就绪方式。

在算法中， $\mathcal{G}_{\text{remote}}$ 包含远端输出 tile， $\mathcal{D}(g)$ 包含 tile g 读取的输入分块。submit 块异步提交工作，本地 GEMM、AllGather 和相互独立的远端 tile 因此可以重叠。Publish 在复制前暴露输入分片，Join 让后续工作等待全部三条路径。上下文按调用顺序复用。

Algorithm 1: 行分块驱动的 AG-GEMM

Input: $A_s, W_s, p, \{C_c\}$, 工作区 X
Output: $D_s = \text{concat}_{u=0}^{p-1}(A_u W_s)$

- 1 InitReady(ready);
- 2 **submit** 本地 GEMM **do**
- 3 $D_{s,s} \leftarrow A_s W_s$;
- 4 **submit** AllGather **do**
- 5 $\hat{A} \leftarrow \text{Publish}(A_s)$;
- 6 **foreach** $(u, c), u \neq s$ **do**
- 7 $X[u, c] \leftarrow \text{Copy}(\hat{A}_u[C_c])$;
- 8 $\text{ready}[u, c] \leftarrow 1$;
- 9 **foreach** $g \in \mathcal{G}_{\text{remote}}$ **do**
- 10 **submit** tile g **do**
- 11 **await** $\bigwedge_{(u,c) \in \mathcal{D}(g)} \text{ready}[u, c] = 1$;
- 12 $D_s[g] \leftarrow \text{TileGEMM}(X, W_s, g)$;
- 13 Join();
- 14 **return** D_s ;

4.2 RS/AR 执行与槽位复用

4.2.1 运行时状态与任务提交. AnchorOverlap 为每条归约链路创建执行上下文, 持有本地 GEMM 结果、输出缓冲、对称暂存槽位和通信流。生产者写入本地结果, 传输流复制待交接的数据, 归约流读取暂存并更新输出。运行时在相应设备流上插入等待, 并在通信与归约之间建立事件依赖。这些等待不需要额外的轮询线程块。通信与归约使用高优先级流, 资源配置为允许生产与消费并发执行。

每个暂存位置有一个到达状态 $\text{arrival}_d[s, x]$ 和一个释放状态 $\text{free}_d[x]$, 分别标识源 s 送达的数据和目的 d 已消费的数据。为区分同一位置上的先后分块, 运行时还维护调用序号基值 β 、RS 就绪轮次 γ 以及数组 $\text{last}[x]$ 。该数组记录位置 x 最近已排定的序号, 下一个分块由它推导需要等待的释放序号, 它并不表示消费已经完成。

对于逻辑分块 $\omega = (c, b)$, 运行时赋予 $\tau_\omega = \beta + cB + b + 1$, 从 $\text{last}[x]$ 取得前驱序号 π_ω , 并将该表项更新为 τ_ω 。例如在深度为二的 RS 窗口中, $(c+2, b)$ 复用 (c, b) 的位置, 因此要等待后者的释放序号。上下文创建时状态从零开始。后续调用保留逐槽位记录并推进 β , RS 同时推进其就绪轮次 γ 。各 rank 使用相互匹配的分块索引和序号区间, 每个上下文按调用顺序复用。

补充材料中的算法 2 给出源端与目的端的操作, 并区分异步提交与完成后的消费。下面的实例说明 RS 和 AR 如何使用这些状态。

4.2.2 RS 前沿生产与窗口化归约. 多带 RS 路径以先后提交的两个 GEMM 内核实现前沿优先生产。第一个内核为所有目的 rank 计算前 F 个行分块, 第二个内核计算其余分块。一个面板的全部输出 tile 写入完成后, 生产者发布其 `chunk_signal`。传输流随后可以复制该面板,

使归约与仍在运行的生产内核重叠。单带配置使用按行分块发布就绪的生产者。

设 $P_{s \rightarrow d}[C_c, B_b]$ 为源 rank s 为目的 d 生成的本地贡献。目的端计算

$$D_d[C_c, B_b] = \sum_{s=0}^{p-1} P_{s \rightarrow d}[C_c, B_b]. \quad (4)$$

源端通信流把每份远端贡献写入目的窗口中源 s 的区域, 并发布 $\text{arrival}_d[s, x] = \tau_\omega$ 。目的端归约器等待本地面板和全部远端源的匹配到达序号, 以本地贡献作为归约初值, 读取并累加远端贡献, 写入输出后置 $\text{free}_d[x] = \tau_\omega$ 。直接使用本地贡献节省一次暂存复制。

AnchorOverlap 按交接顺序将面板轮转分配给 R_{RS} 条归约流水线。每条流水线等待自身的任务依赖, 窗口位置在消费后即可供后续面板使用。流水线数控制同时推进的任务数量, 窗口深度控制驻留数据, 二者分别配置。

4.2.3 AR 前沿交接与递归倍增. 对于每个 AR 前沿分块, AnchorOverlap 向计算流提交 GEMM 并立即向归约流提交消费, 遵循第 3.2 节定义的公共分块顺序。当前 cuBLAS 路径将各分块 GEMM 写入预分配的本地结果缓冲, 归约流把每个结果复制到其输出累积区域。

归约从 $Z_s^{(0)} = P_s[C_c, B_b]$ 出发。对于二的幂的组规模, 在第 $j \in [0, \log_2 p]$ 轮, rank s 与 $v = s \oplus 2^j$ 交换当前累积结果, 其中 $\oplus$ 表示按位异或:

$$Z_s^{(j+1)} = Z_s^{(j)} + Z_v^{(j)}. \quad (5)$$

经过 $\log_2 p$ 轮后, 每个 rank 持有 $D_s[C_c, B_b]$, 即全部 p 份贡献 $P_u[C_c, B_b]$ 之和。四卡时的两轮对应图 3(b) 中的 reduce1 和 reduce2。它们顺序执行并与后续分块的 GEMM 重叠。

每次交换中, 发送流等待当前累积结果和对端槽位的释放, 随后把结果复制到对端属于本 rank 的源区域。本地归约器同时等待出站复制完成和入站数据到达后才原地累加, 因此不会改写仍在被发送读取的数据。最后一轮结束后, 它发布该槽位的释放序号并记录一个本地分块完成事件。

AnchorOverlap 用这些事件限制生产超前消费的距离。设生产超前为 L_{prod} 个行分块、输出分块数为 n , 允许的在途分块数为 $W_{\text{look}} = \min(R_{AR}, L_{\text{prod}}B, n)$ 。在提交第 $k \geq W_{\text{look}}$ 个分块之前, 计算流先等待第 $k - W_{\text{look}}$ 个分块的本地完成事件。释放序号继续保护远端槽位。

4.3 考虑约束的配置搜索

消费者更早启动并不必然使最小分块或最高通信并发性成为最快配置。小分块带来更多交接机会, 也带来更多通知和任务提交。更多归约资源可以加速消费, 但会挤占 GEMM 的资源。浅窗口使传输更容易等待释放, 深窗口增加暂存占用。这些权衡随矩阵形状变化, 因此 AnchorOverlap 为每个形状选择一个配置并在后续调用中复用。

搜索在可执行候选中寻找较低的端到端集合通信-GEMM 时延。对于算子 o 、形状 $x = (M, N, K, p)$ 和硬件

环境 H ，一个候选为

$$\theta_o = (h_c, w_b, \ell, \rho, \chi_o), \quad (6)$$

其中 h_c, w_b 决定交接数据量， ℓ, ρ 控制通信并发和消费者 SM 配额， χ_o 保存链路特定设置。RS 使用前沿深度、窗口深度和流水线数，AR 使用分块槽数、生产超前和 GEMM 后端。完整测量所有组合需要大量运行，因此搜索把结构选择与资源细化分开。

补充材料中的算法 3 给出两阶段搜索。以 RS 候选组为例，搜索器先规范化窗口深度、前沿深度与任务数之间的关系，去重后按 (h_c, L, B) 分组，再依据窗口与归约队列的比例、通信通道数和前沿占窗口的比例等启发式，从每组保留少量代表。启发式规则减少需要测量的候选数量，候选性能仍由全链路时延决定。运行预算在结构筛查和资源细化之间分配。

第一阶段以初始的稳态和尾部 SM 配额对结构候选做短测，按各 rank 最大总时延排序。第二阶段取较快的结构并测量不同的稳态与尾部 SM 配额，避免对每种结构做完整资源扫描。搜索器随后取第二阶段成功执行中最快的若干候选，提高预热和测量次数，在长测成功的候选中选择时延最低的配置。若全部长测失败，脚本保留粗测最优项作为尚未通过长测验证的回退。

候选运行包含实际工作区分配和算子执行。分配失败或执行失败的候选不进入有效时延排序。对称内存预算要求

$$M_o^{\text{data}}(\theta) + M_o^{\text{meta}}(\theta) \leq M_{\text{sym}}^{\text{budget}}, \quad o \in \{\text{RS}, \text{AR}\}, \quad (7)$$

其中数据项是式 (1) 和式 (2) 的接收暂存，元数据项覆盖状态数组和对齐开销。共享的预分配工作区按其预留尺寸计入。该预算是必要可行性条件，实际分配和执行决定候选是否产生计时记录。

对于 AG，AnchorOverlap 改为为本地 GEMM 和远端消费者选择内核配置，使两条路径都适配输入形状。

5 实验评估

我们在算子、TP 层和端到端预填充三个层级评估 AnchorOverlap，随后通过消融分析考察就绪粒度、前沿优先调度、活动窗口暂存和配置搜索。所有加速比均在同一平台内以匹配的基线归一化。

5.1 实验设置

5.1.1 平台与软件. 主服务器配备四块 NVIDIA A100-SXM4-80GB GPU，任意两卡之间为 NVLink NV12 互连，运行 Ubuntu 24.04 (Linux 5.15) 和驱动 580.65.06，软件为 Python 3.12、PyTorch 2.7.0 (CUDA 12.6)、NCCL 2.26.2、Triton 3.4.0 和 NVSHMEM 3.3.9。第二台服务器配备四块海光 K100_AI-64GB 加速卡，运行 DTK 26.04、Python 3.11、PyTorch 2.7.1、Triton 3.1.0、RCCL 2.22.3 和 DUSHMEM 3.2.5。DUSHMEM 提供 NVSHMEM 对应的 DCU 侧单侧对称内存原语，使相同的就绪、序号和活动窗口协议语义可以在该平台实例化。A100 覆盖全部算子链路、层集成和主要机制消融，K100_AI 覆盖算子和模型级预填充。

5.1.2 负载、基线与计时. 算子实验覆盖全部三条链路。模型级实验覆盖 Llama3-70B 与 Qwen2.5-72B 的 TP Attention、TP MLP 和端到端预填充，配置为 TP=4、BF16、batch 64、序列长度 128，因此 $M = 8192$ 。计时的端到端路径覆盖 token 嵌入、全部 Transformer 层、最终归一化和 LM 头词表投影，返回最后一个预填充 token 的 logits。两个模型的 MLP (M, N, K) 形状分别为 $(8192, 28672, 8192)$ 和 $(8192, 29568, 8192)$ 。

Attention 和 MLP 每次启动预热 20 次并对 100 次图重放取均值，端到端预填充预热 10 次并对 20 次计时重放取均值。每条路径的计时区间由 CUDA 同步界定，Torch 段与 Triton 段之间以 NVSHMEM barrier 隔离。预热后用 CUDA 事件计量重放循环，同步该流并将总事件时间除以迭代次数。编译和图捕获不计入。每个模型级条件保留一次最终启动的加速比。

算子图使用串行集合通信-GEMM 基线归一化，A100 上以 PyTorch/NCCL 实现，K100_AI 上以 PyTorch/RCCL 实现。可比较的方法为 AG 上的 FLUX 和 Triton-distributed，RS 上的 FLUX、FlashOverlap 和 Triton-distributed，AR 上的 FlashOverlap 和 Triton-distributed。AG 就绪研究以 Triton-distributed 归一化，RS 前沿研究以匹配的均匀调度实现归一化。

模型级预填充图报告每条记录路径相对语义等价 Torch 不重叠参考的加速比：

$$\text{Speedup}_{\text{torch}} = \frac{T_{\text{torch}}}{T_{\text{variant}}}. \quad (8)$$

5.1.3 正确性与槽位复用验证. 正确性检查与性能测量使用相互独立的执行。性能图只采用通过正确性检查并完成计时的配置记录。模型级 Attention、MLP 和端到端检查均使用 $(\text{atol}, \text{rtol}) = (2 \times 10^{-2}, 2 \times 10^{-2})$ 。端到端正确性对 logits 施加 FP32 softmax 后与 Torch/HuggingFace 参考比较概率，softmax 与比较均在计时区间之外。

算子检查将每个 rank 的输出与语义等价的串行 Torch 实现比较，全部 rank 满足容差才算通过。表 2 报告各链路容差和启动结果。为考察反复槽位复用，RS 窗口扫描对每个 segment 使用八个逻辑分块并取 $L \in \{1, 2, 4, 8\}$ ，深度 1、2、4 强制不同分块复用同一物理槽位。AG 检查覆盖从 2048 到 256 行的就绪粒度。

5.2 算子级性能

图 7 和图 8 分别比较 A100 和 K100_AI 上三条链路的算子性能，均以同平台串行基线归一化。

如图 7 所示，在四个测试形状上，AnchorOverlap 各路径相对串行基线在 A100 上达到 AG 0.98–1.16×、RS 1.02–1.27×、AR 0.99–1.19×。如图 8 所示，同样形状在 K100_AI 上达到 AG 1.08–1.35×、RS 1.06–1.40×、AR 1.06–1.23×，全文最高的 1.40× 出现在 RS 形状 $(8192, 29568, 8192)$ 上。

FlashOverlap 的调优搜索无法在 $(8192, 29568, 8192)$ 上完成 RS 和 AR 链路，两条路径在评估第一个候选时即报错中止，原因是候选 tile 形状无法整除矩阵维度。

5.2.1 可行性对比. 默认约 1 GiB/PE 的对称堆使 Triton-distributed 在多个 RS/AR 形状上分配失败。将预算提高

Table 2. 链路级正确性与启动结果，含开发期运行。每个 BF16 阈值同时适用于绝对和相对容差。

链路	BF16 容差	复用压力与通过记录
AG-GEMM	10^{-3}	121/129 次启动通过，八次失败来自同一形状的两个早期调试批次。
GEMM-RS	2×10^{-2}	强制循环复用，窗口扫描 60/60 通过，受控顺序 11/12 完成（一次数值检查通过后计时超时）。
GEMM-AR	2×10^{-2}	强制循环复用，窗口扫描 60/60 通过，受控顺序 12/12 完成。

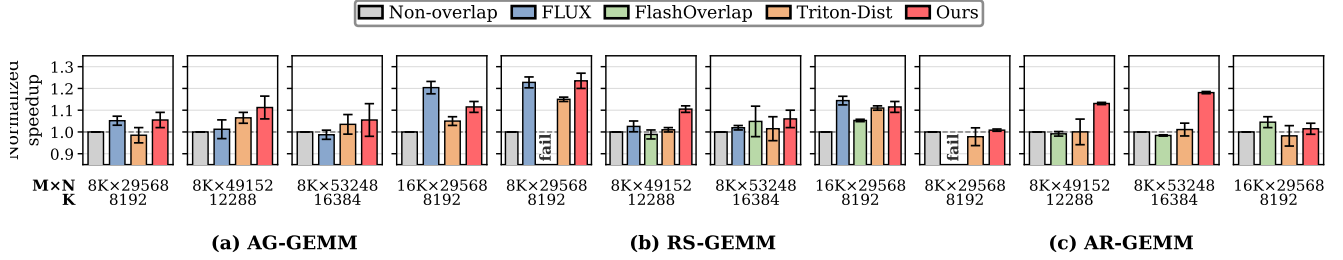


Figure 7. 四张 A100 GPU 上的算子级加速比，三条集合通信-GEMM 链路分别以各自匹配的串行基线归一化。

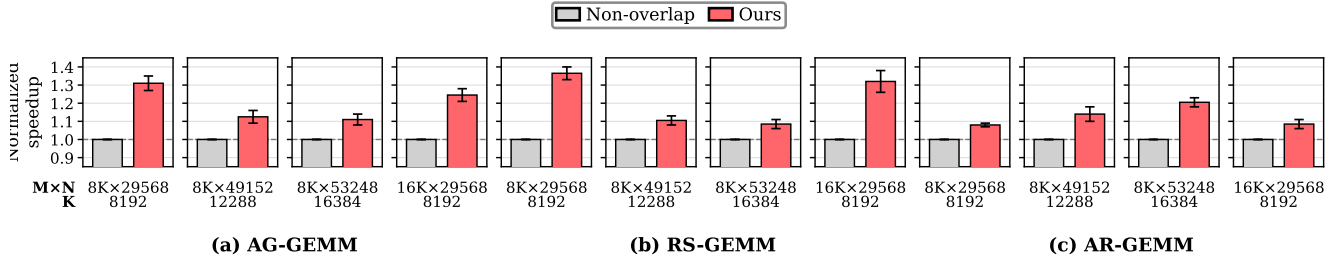


Figure 8. 四张 K100_AI 加速卡上的算子级加速比，三条集合通信-GEMM 链路分别以各自匹配的串行基线归一化。

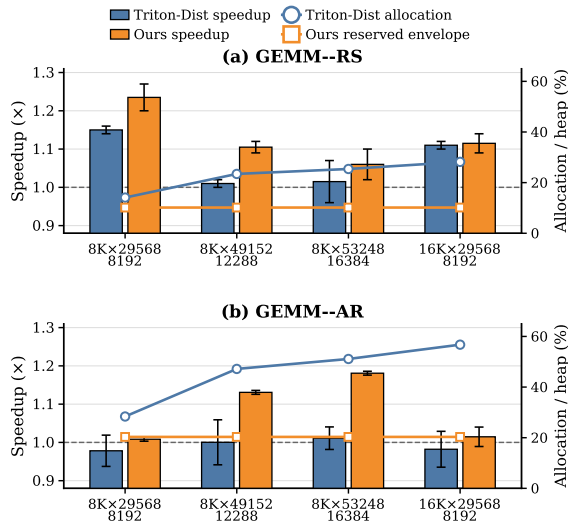


Figure 9. A100 上 GEMM-RS/AR 的加速比与对称内存分配。右轴百分比以 8 GiB/PE 堆预算为分母，对应 Triton-distributed 的实测分配量与 AnchorOverlap 的固定窗口预留。

到 8 GiB/PE 后可以完成全部四个形状。图 9 将扩容后的加速比与对称暂存需求并列，折线报告 Triton-distributed 的实测分配量和 AnchorOverlap 的固定窗口预留占堆预算的比例。Triton-distributed 的分配量在 RS 上从 1.13 增至 2.26 GiB/PE，在 AR 上从 2.27 增至 4.54 GiB/PE。AnchorOverlap 在这些配置下为 RS 预留 0.81 GiB/PE、为 AR 预留 1.63 GiB/PE，使四个形状共享一个固定容量的暂存窗口。

为考察缓冲需求随工作量的扩展，我们把对称堆预算提高到 32 GiB，并在图 10 中比较两个 RS 实现的对称内存分配随矩阵行数的增长，固定 $N=53248$ 和 $K=16384$ 。若每个逻辑单元永久占有一个对称暂存位置，分配量随逻辑工作集增长，不复用基线从 $M=8192$ 的 2.03 GiB 线性增长， $M=131072$ 时所需的 32.5 GiB 超出预算并分配失败。AnchorOverlap 以固定 $L=4$ 活动窗口保持在约 0.81 GiB，把对称暂存占用与逻辑工作量解耦，在 $M=65536$ 处拉开约 20× 差距。

5.3 模型级预填充评估

模型执行包含注意力、激活、归一化、LM 头等集合通信-GEMM 路径之外的工作。我们因此检验算子收益能

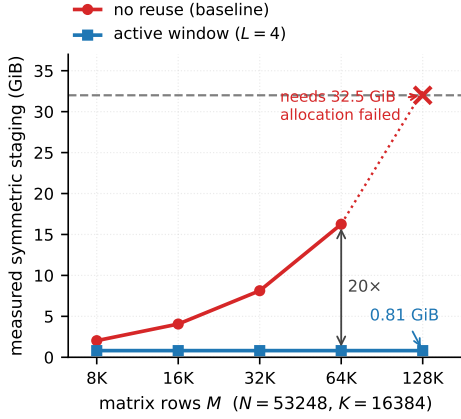


Figure 10. 对称暂存分配与需求随矩阵维度 M 的变化。在 $N=53248$ 、 $K=16384$ 和 32 GiB/PE 堆上限下，不复用基线在 $M=131072$ 时需要 32.5 GiB/PE 并分配失败， $L=4$ 活动窗口保持在 0.81 GiB/PE。

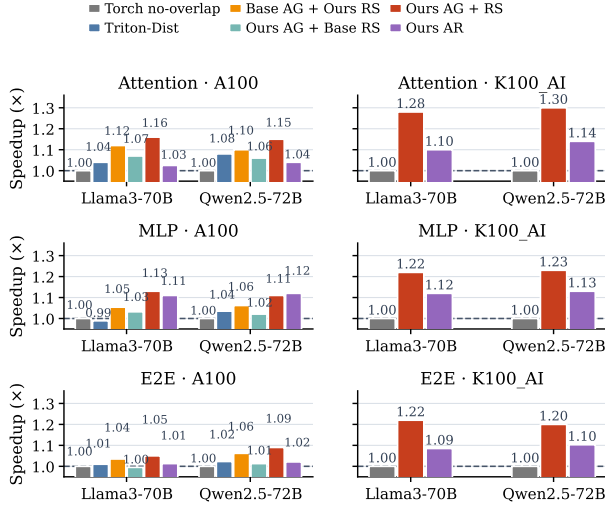


Figure 11. TP Attention、TP MLP 与端到端预填充的加速比。

否带入 TP Attention、TP MLP 和包含 LM 头的端到端预填充，同时报告 AG/RS 组合路径和 AR 路径。

5.3.1 TP Attention、TP MLP 与端到端预填充. 图 11 对每个模型、平台和受测路径各含一条计时记录。在 A100 上，Ours AG/Ours RS 组合路径在 Llama3-70B/Qwen2.5-72B 上使 Attention 达到 1.16 $\times$ /1.15 $\times$ ，MLP 达到 1.13 $\times$ /1.11 $\times$ ，端到端预填充达到 1.05 $\times$ /1.09 $\times$ 。Ours AR 的端到端预填充为 1.01 $\times$ /1.02 $\times$ 。Triton-distributed 和仅替换 AG 或仅替换 RS 的变体表明，两条链路在不同负载下的贡献不同。

在 K100_AI 上，端到端预填充在 Ours AG/RS 下达到 1.20–1.22 $\times$ ，在 Ours AR 下达到 1.09–1.10 $\times$ 。

Table 3. 考虑约束的 GEMM-RS 搜索成本与最终验证，形状 $8192 \times 29568 \times 8192$ 。

候选数	方法	测量	时延	配置
64	穷举	67	5.49 ms	(512,4,4,6,12,2,2,1)
64	两阶段	13	5.46 ms	(512,4,4,6,16,2,2,1)
128	穷举	131	5.39 ms	(512,4,4,6,24,2,2,1)
128	两阶段	15	5.47 ms	(512,4,4,6,12,2,2,1)

注：所有搜索均完成 top-3 验证。配置依次为 *chunk_rows*、*active_chunk_window*、*stage_slots*、*steady_sms*、*tail_sms*、*comm_lanes*、*n_bands*、*frontier_chunks*。

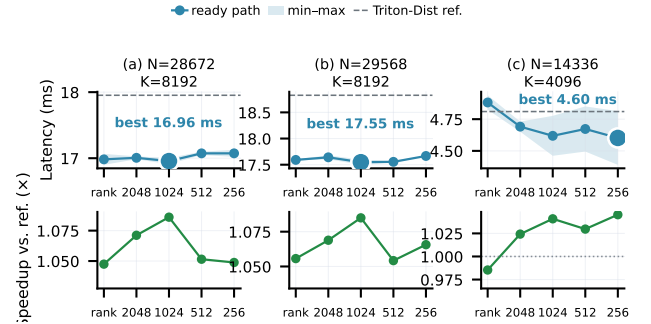


Figure 12. $M = 28672$ 时三个 (N, K) 形状上的 AG-GEMM 就绪粒度。上排为 rank 最大时延与形状内中位参考，下排为相对实测 Triton-distributed 参考的逐格加速比。

5.4 消融分析

我们用消融实验考察就绪粒度、前沿优先调度和活动窗口暂存。

5.4.1 细粒度就绪单元. AG 研究比较 rank 就绪与四种固定行 tile 就绪粒度，指标为 rank 最大时延和相对 Triton-distributed 的加速比。图 12 展示了通信计算比和消息大小各不相同的三组实测形状。

粗粒度就绪推迟消费者启动，过细的就绪则可能以信号、轮询和调度开销抵消重叠收益。在图 12 中，两个较大 N 形状在 1024 行就绪处达到峰值 (1.086 $\times$ 和 1.085 $\times$)，较小的 N, K 形状在 256 行处达到峰值 (1.045 $\times$)，其中 rank 就绪甚至低于参考 (0.985 $\times$)。合适的粒度随负载变化，更细的粒度并不总会带来更好的性能。

5.4.2 消费者锚定的前沿调度. 为检验优先处理消费者唤醒工作能否改善启动和完整链路时延，我们在匹配形状和运行时资源下比较均匀与前沿优先的 GEMM-RS 路

径。研究使用三个形状，(8192, 29568, 8192) 是 Qwen2.5-72B TP 预填充的 MLP 投影，(8192, 49152, 12288) 和 (8192, 53248, 16384) 覆盖更宽或更深的负载。

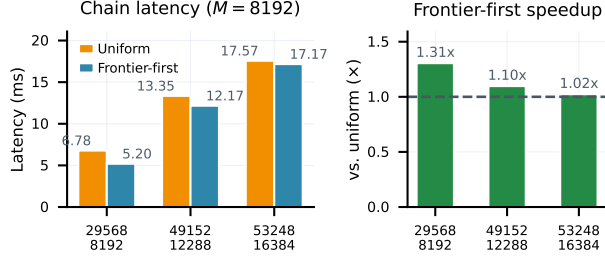


Figure 13. 匹配的 GEMM-RS 实现分别为均匀顺序与消费者锚定的前沿优先顺序。

前沿优先路径在三个形状上均取得更低的完整链路时延（图 13），相对均匀调度的加速比为 1.02–1.31 $\times$ 。最大增益出现在 (8192, 29568, 8192)，(8192, 53248, 16384) 为 1.02 $\times$ 。这些结果表明，更早供给归约输入可以改善完整链路时延，增益幅度随矩阵形状变化。

5.4.3 活动窗口对称暂存. 窗口研究使用同样的三个 GEMM-RS 形状，每个形状含八个逻辑分块。深度 $L = 8$ 同时容纳全部分块，作为全驻留参考。我们在每个 $L \in \{1, 2, 4, 8\}$ 下执行五次独立启动，共完成 60 次启动。图 14 报告相对全驻留的中位时延和对称暂存占用，包含数据槽位和到达释放序号数组。

更小的窗口减少驻留容量，但使后续分块更容易等待尚未释放的槽位。 $L = 1$ 时占用下降 87.5%，三个形状的中位时延增加 4.86%–20.85%。 $L = 2$ 时占用下降 75%，时延增加 0.67%–12.06%。 $L = 4$ 时占用减半，中位时延仅增加 0.41%–0.58%。

半深度窗口在三个测试形状上保持接近全驻留的性能，进一步压缩容量将带来更大且随形状变化的时延代价。

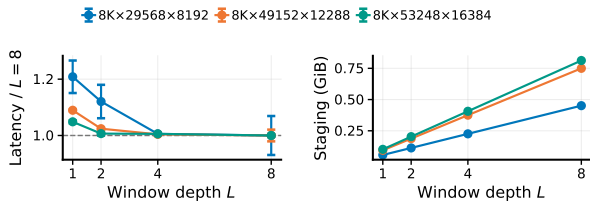


Figure 14. 三个案例形状上不同活动窗口深度的 GEMM-RS 时延与暂存权衡。

5.5 考虑约束的配置搜索

表 3 在 (8192, 29568, 8192) 上比较两阶段与穷举 GEMM-RS 搜索，候选空间分别为 64 和 128。搜索成本以搜索和最终验证的测量次数衡量，配置质量以最终验证中所有 rank 的最大链路时延衡量。两种方法都对前三名候选做验证。在该形状上，两阶段搜索最多以 1.48% 的最终时延换取 80.6% 和 88.5% 的测量预算缩减。

6 结论

本文提出 AnchorOverlap，一套面向集合通信-GEMM 链路的运行时执行协议。其核心思想是，依据消费者启动所需的最小数据单元推导交接粒度，采用与链路相适应的生产和交接方式，并用到达与释放票据保护单侧槽位复用。

我们在两个四卡加速平台上实现了 AG-GEMM、GEMM-RS 和 GEMM-AR。该协议相对匹配的串行基线最高取得 1.40 $\times$ 的算子级加速。循环复用最多将实测对称暂存占用降为全驻留的八分之一， $L = 4$ 在中位时延仅增加 0.4%–0.6% 的前提下使占用减半，说明这些链路无需为每个逻辑分块保留暂存即可维持接近全驻留的性能。AG/RS 的收益也可以带入包含 LM 头的端到端预填充 (1.05 $\times$ –1.22 $\times$)。

References

- [1] Anirudha Agrawal, Shaizeen Aga, Suchita Pati, and Mahzabeen Islam. 2025. ConCCL: Optimizing ML Concurrent Computation and Communication with GPU DMA Engines. In *2025 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 1–11. doi:10.1109/ISPASS64960.2025.00018
- [2] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, and Ramachandran Ramjee. 2023. Sarathi: Efficient LLM Inference by Piggybacking Decodes with Chunked Prefills. *arXiv preprint arXiv:2308.16369* (2023).
- [3] Muhammad Awad, Muhammad Osama, and Brandon Potter. 2025. Iris: First-Class Multi-GPU Programming Experience in Triton. *arXiv preprint arXiv:2511.12500* (2025). <https://arxiv.org/abs/2511.12500>
- [4] Li-Wen Chang, Wenlei Bao, Qi Hou, Chengquan Jiang, Ningxin Zheng, Yinmin Zhong, Xuanrun Zhang, Zuquan Song, Chengji Yao, Ziheng Jiang, Haibin Lin, Xin Jin, and Xin Liu. 2024. FLUX: Fast Software-Based Communication Overlap on GPUs Through Kernel Fusion. *arXiv preprint arXiv:2406.06858* (2024).
- [5] Chang Chen, Xiuhong Li, Qianchao Zhu, Jiangfei Duan, Peng Sun, Xingcheng Zhang, and Chao Yang. 2024. Centauri: Enabling Efficient Scheduling for Communication-Computation Overlap in Large Model Training via Communication Partitioning. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Vol. 3. 178–191. doi:10.1145/3620666.3651379
- [6] Meghan Cowan, Saeed Maleki, Madanlal Musuvathi, Olli Saarikivi, and Yifan Xiong. 2023. MSCCLang: Microsoft Collective Communication Language. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Vol. 2. 502–514. doi:10.1145/3575693.3575724
- [7] Minyu Cui, Anna Wingkvist, and Morgan Ericsson. 2026. Fine-grained Computation-Communication Overlap via Tile-level Signaling and Scheduling for Mixture-of-Experts. *arXiv preprint arXiv:2607.19539* (2026). <https://arxiv.org/abs/2607.19539> To appear at ICPP 2026.
- [8] DeepSeek-AI et al. 2026. DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence. *arXiv preprint arXiv:2606.19348* (2026).
- [9] Jiarui Fang and Shangchun Zhao. 2024. USP: A Unified Sequence Parallelism Approach for Long Context Generative AI. *arXiv preprint arXiv:2405.07719* (2024).
- [10] Raja Gond, Nipun Kwatra, and Ramachandran Ramjee. 2026. TokenWeave: Efficient Compute-Communication Overlap for Distributed LLM Inference. In *Proceedings of the 9th Conference on Machine Learning and Systems (MLSys)*. arXiv:2505.11329 <https://openreview.net/forum?id=rh2Ylffkq6>
- [11] Ke Hong, Guohao Dai, Jiaming Xu, Qiuli Mao, Xiuhong Li, Jun Liu, Kangdi Chen, Yuhan Dong, and Yu Wang. 2024. FlashDecoding++: Faster Large Language Model Inference with Asynchronization, Flat GEMM Optimization, and Heuristics. In *Proceedings of Machine Learning and Systems*, Vol. 6. 148–161. https://proceedings.mlsys.org/paper_files/paper/2024/hash/5321b1dabcd2be188d796c21b733e8c7-Abstract-Conference.html
- [12] Ke Hong, Xiuhong Li, Minxu Liu, Qiuli Mao, Tianqi Wu, Zixiao Huang, Lufang Chen, Zhong Wang, Yichong Zhang, Zhenhua Zhu, Guohao Dai, and Yu Wang. 2026. Efficient and Adaptable Overlapping for Computation and Communication via Signaling and Reordering. In *Proceedings of the 21st European Conference on Computer Systems (EuroSys)*. 1894–1911. doi:10.1145/3767295.3769370
- [13] Abhinav Jangda, Jun Huang, Guodong Liu, Amir Hossein Nodehi Sabet, Saeed Maleki, Youshan Miao, Madanlal Musuvathi, Todd Mytkowicz, and Olli Saarikivi. 2022. Breaking the Computation and Communication Abstraction Barrier in Distributed Machine Learning Workloads. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 402–416. doi:10.1145/3503222.3507778
- [14] Ziheng Jiang, Haibin Lin, Yinmin Zhong, Qi Huang, Yangrui Chen, Zhi Zhang, Yanghua Peng, Xiang Li, Cong Xie, Shibiao Nong, Yulu Jia, Sun He, Hongmin Chen, Zhihao Bai, Qi Hou, Shipeng Yan, Ding Zhou, Yiyao Sheng, Zhuo Jiang, Haohan Xu, Haoran Wei, Zhang Zhang, Pengfei Nie, Leqi Zou, Sida Zhao, Liang Xiang, Zherui Liu, Zhe Li, Xiaoying Jia, Jianxi Ye, Xin Jin, and Xin Liu. 2024. MegaScale: Scaling Large Language Model Training to More Than 10,000 GPUs. In *Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. 745–760. <https://www.usenix.org/conference/nsdi24/presentation/jiang-ziheng>
- [15] Rezaul Karim, Austin Wen, Wang Zongzuo, Weiwei Zhang, Yang Liu, and Walid Ahmed. 2026. CommFuse: Hiding Tail Latency via Communication Decomposition and Fusion for Distributed LLM Training. *arXiv preprint arXiv:2604.24013* (2026).
- [16] Kimi Team et al. 2026. Kimi K3: Open Frontier Intelligence. *arXiv preprint arXiv:2607.24653* (2026). <https://arxiv.org/abs/2607.24653>

- [17] Vijay Anand Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. 2022. Reducing Activation Recomputation in Large Transformer Models. *arXiv preprint arXiv:2205.05198* (2022).
- [18] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*. 611–626. doi:10.1145/3600006.3613165
- [19] Seonho Lee, Jihwan Oh, Seokjin Go, and Divya Mahajan. 2025. Characterizing Compute-Communication Overlap in GPU-Accelerated Distributed Deep Learning: Performance and Power Implications. In *Proceedings of the IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 353–355. doi:10.1109/ISPASS64960.2025.00041
- [20] Shengwei Li, Zhiquan Lai, Dongsheng Li, Yanqi Hao, Weijie Liu, Keshi Ge, Xiaoge Deng, and Kai Lu. 2025. Oases: Efficient Large-Scale Model Training on Commodity Servers via Overlapped and Automated Tensor Model Parallelism. *IEEE Transactions on Parallel and Distributed Systems (TPDS)* 36, 9 (2025), 1828–1840. doi:10.1109/TPDS.2025.3583165
- [21] Hao Liu, Matei Zaharia, and Pieter Abbeel. 2024. Ring Attention with Blockwise Transformers for Near-Infinite Context. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*. <https://openreview.net/forum?id=WsRHpHH4s0>
- [22] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R. Devanur, Gregory R. Ganger, Phillip B. Gibbons, and Matei Zaharia. 2019. PipeDream: Generalized Pipeline Parallelism for DNN Training. In *Proceedings of the 27th Symposium on Operating Systems Principles (SOSP)*. 1–15. arXiv:1806.03377 doi:10.1145/3341301.3359646
- [23] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Anand Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. 2021. Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*. 1–15. doi:10.1145/3458817.3476209
- [24] NVIDIA. 2024. cuBLASmp: Parallel GEMM and Communication. <https://docs.nvidia.com/cuda/cublasmp/>.
- [25] NVIDIA. 2024. NCCL: NVIDIA Collective Communication Library. <https://github.com/NVIDIA/nccl>.
- [26] NVIDIA. 2024. NVSHMEM: OpenSHMEM-based PGAS for GPU. <https://developer.nvidia.com/nvshmem>.
- [27] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Ínigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA)*. 118–132. doi:10.1109/ISCA59077.2024.00019
- [28] Suchita Pati, Shaizeen Aga, Mahzabeen Islam, Nuwan Jayasena, and Matthew D. Sinclair. 2024. T3: Transparent Tracking & Triggering for Fine-grained Overlap of Compute & Collectives. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Vol. 2. 1146–1164. doi:10.1145/3620665.3640410
- [29] Ramya Prabhu, Ajay Nayak, Jayashree Mohan, Ramachandran Ramjee, and Ashish Panwar. 2025. vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 1133–1150. doi:10.1145/3669940.3707256
- [30] PyTorch Foundation and TorchTitan Team. 2024. Async Tensor Parallelism (Async-TP). <https://discuss.pytorch.org/t/distributed-w-torchtitan-introducing-async-tensor-parallelism-in-pytorch/209487>.
- [31] Xinwei Qiang, Yue Guan, Zhengding Hu, Keren Zhou, Yufei Ding, and Adnan Aziz. 2026. Syncopate: Efficient Multi-GPU AI Kernels via Automatic Chunk-Centric Compute-Communication Overlap. In *Proceedings of the 20th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, Seattle, WA, 331–347. arXiv:2601.20595 <https://www.usenix.org/conference/osdi26/presentation/qiang>
- [32] Qwen Team. 2026. Qwen3.8-Max: A New Bar for Coding and Cowork. <https://qwen.ai/blog?id=qwen3.8>
- [33] Aashaka Shah, Vijay Chidambaram, Meghan Cowan, Saeed Maleki, Madan Musuvathi, Todd Mytkowicz, Jacob Nelson, Olli Saarikivi, and Rachee Singh. 2023. TACCL: Guiding Collective Algorithm Synthesis using Communication Sketches. In *Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. 593–612. <https://www.usenix.org/conference/nsdi23/presentation/shah>
- [34] Baodi Shan, Mauricio Araya-Polo, and Barbara Chapman. 2026. GICC: A High-Performance Runtime for GPU-Initiated Communication and Coordination in Modern HPC Systems. In *Proceedings of the 35th International Symposium on High-Performance Parallel and Distributed Computing (HPDC)*. 45–57. doi:10.1145/3806645.3807576
- [35] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. *arXiv*

- preprint arXiv:1909.08053* (2019).
- [36] Weigao Sun, Zhen Qin, Weixuan Sun, Shidi Li, Dong Li, Xuyang Shen, Yu Qiao, and Yiran Zhong. 2024. CO2: Efficient Distributed Training with Full Communication-Computation Overlap. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*. <https://openreview.net/forum?id=ZO5cn4lfaN>
 - [37] Guanhua Wang, Shivaram Venkataraman, Amar Phanishayee, Jorgen Thelin, Nikhil Devanur, and Ion Stoica. 2020. Blink: Fast and Generic Collectives for Distributed ML. In *Proceedings of Machine Learning and Systems*, Vol. 2. 172–186. https://proceedings.mlsys.org/paper_files/paper/2020/hash/cd3a9a55f7f3723133fa4a13628cdf03-Abstract.html
 - [38] Guanhua Wang, Chengming Zhang, Zheyu Shen, Ang Li, and Olatunji Ruwase. 2024. Domino: Eliminating Communication in LLM Training via Generic Tensor Slicing and Overlapping. *arXiv preprint arXiv:2409.15241* (2024).
 - [39] Shibo Wang, Jinliang Wei, Amit Sabne, Andy Davis, Berkin Ilbeyi, Blake Hechtman, Dehao Chen, Karthik Srinivasa Murthy, Marcello Maggioni, Qiao Zhang, Sameer Kumar, Tongfei Guo, Yuanzhong Xu, and Zongwei Zhou. 2023. Overlap Communication with Dependent Computation via Decomposition in Large Deep Learning Models. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Vol. 1. 93–106. doi:10.1145/3567955.3567959
 - [40] Jianwen Xian, Zhiyuan Xu, Yuchen Li, Ziliang Lai, Kang He, Zhen Huang, Aichen Feng, Jinyan Chen, Yilin Zhang, Qinqin Chen, and Chengru Song. 2026. X-Stage: An Overlooked Pipeline Stage for Communication-Computation Overlap in DiT Inference. *arXiv preprint arXiv:2607.23264* (2026). <https://arxiv.org/abs/2607.23264>
 - [41] Bin Xiao and Lei Su. 2024. ISO: Overlap of Computation and Communication within Sequence For LLM Inference. *arXiv preprint arXiv:2409.11155* (2024).
 - [42] Guanbin Xu, Zhihao Le, Yinhe Chen, Zhiqi Lin, Zewen Jin, Youshan Miao, and Cheng Li. 2025. AutoCCL: Automated Collective Communication Tuning for Accelerating Distributed and Parallel DNN Training. In *Proceedings of the 22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. 667–683. <https://www.usenix.org/conference/nsdi25/presentation/xu-guanbin>
 - [43] Shulai Zhang, Ningxin Zheng, Haibin Lin, Ziheng Jiang, Wenlei Bao, Chengquan Jiang, Qi Hou, Weihao Cui, Size Zheng, Li-Wen Chang, Quan Chen, and Xin Liu. 2025. Comet: Fine-grained Computation-communication Overlapping for Mixture-of-Experts. In *Proceedings of the 8th Conference on Machine Learning and Systems (MLSys)*. <https://openreview.net/forum?id=fGgQS5VW09>
 - [44] Size Zheng, Wenlei Bao, Qi Hou, Xuegui Zheng, Jin Fang, Chenhui Huang, Tianqi Li, Haojie Duanmu, Renze Chen, Ruifan Xu, Yifan Guo, Ningxin Zheng, Ziheng Jiang, Xinyi Di, Dongyang Wang, Jianxi Ye, Haibin Lin, Li-Wen Chang, Liqiang Lu, Yun Liang, Jidong Zhai, and Xin Liu. 2025. Triton-distributed: Programming Overlapping Kernels on Distributed AI Systems with the Triton Compiler. *arXiv preprint arXiv:2504.19442* (2025).
 - [45] Size Zheng, Jin Fang, Xuegui Zheng, Qi Hou, Wenlei Bao, Ningxin Zheng, Ziheng Jiang, Dongyang Wang, Jianxi Ye, Haibin Lin, Li-Wen Chang, and Xin Liu. 2025. TileLink: Generating Efficient Compute-Communication Overlapping Kernels using Tile-Centric Primitives. In *Proceedings of the Eighth Conference on Machine Learning and Systems (MLSys)*. <https://openreview.net/forum?id=ccjvBkTRRe>
 - [46] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. <https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin>